Solution: Longest Bitonic Subsequence

Problem Statement

Given a number sequence, find the length of its Longest Bitonic Subsequence (LBS). A subsequence is considered bitonic if it is monotonically increasing and then monotonically decreasing.

Example 1:

```
Input: {4,2,3,6,10,1,12}
Output: 5
Explanation: The LBS is {2,3,6,10,1}.
```

Example 2:

```
Input: {4,2,5,9,7,6,10,3,1}
Output: 7
Explanation: The LBS is {4,5,9,7,6,3,1}.
```

Basic Solution

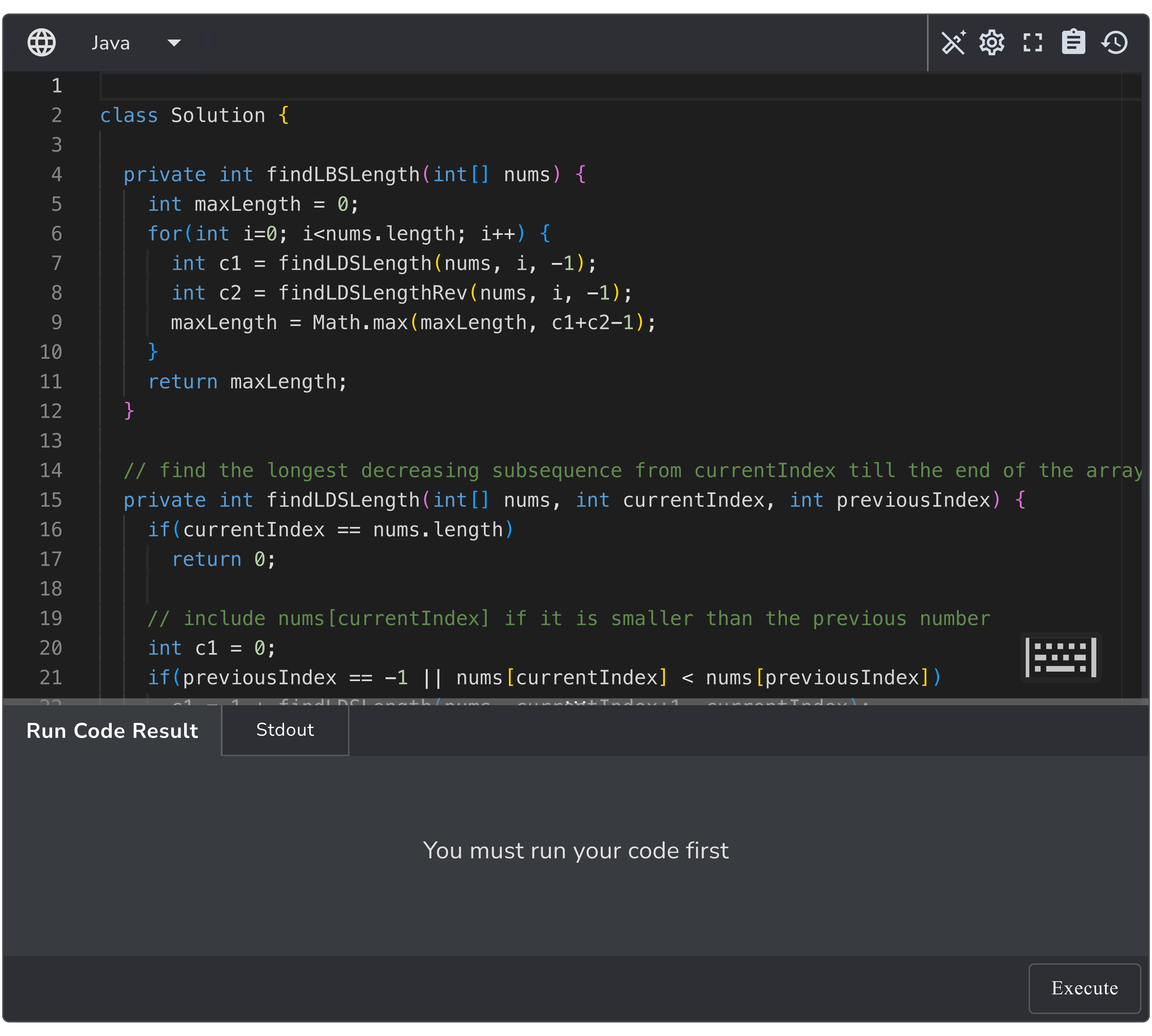
A basic brute-force solution could be to try finding the Longest Decreasing Subsequences (LDS), starting from every number in both directions. So for every index 'i' in the given array, we will do two things:

1. Find LDS starting from 'i' to the end of the array.
2. Find LDS starting from 'i' to the beginning of the array.

LBS would be the maximum sum of the above two subsequences.

Code

Here is the code:



```java

class Solution {

  private int findLBSLength(int[] nums) {
    int maxLength = 0;
    for(int i=0; i<nums.length; i++) {
      int c1 = findLDSLength(nums, i, -1);
      int c2 = findLDSLengthRev(nums, i, -1);
      maxLength = Math.max(maxLength, c1+c2-1);
    }
    return maxLength;
  }

  // find the longest decreasing subsequence from currentIndex till the end of the array
  private int findLDSLength(int[] nums, int currentIndex, int previousIndex) {
    if(currentIndex == nums.length)
      return 0;

    // include nums[currentIndex] if it is smaller than the previous number
    int c1 = 0;
    if(previousIndex == -1 || nums[currentIndex] < nums[previousIndex])
```

You must run your code first

The time complexity of the above algorithm is exponential $O(2^n)$, where 'n' is the lengths of the input array. The space complexity is $O(n)$ which is used to store the recursion stack.

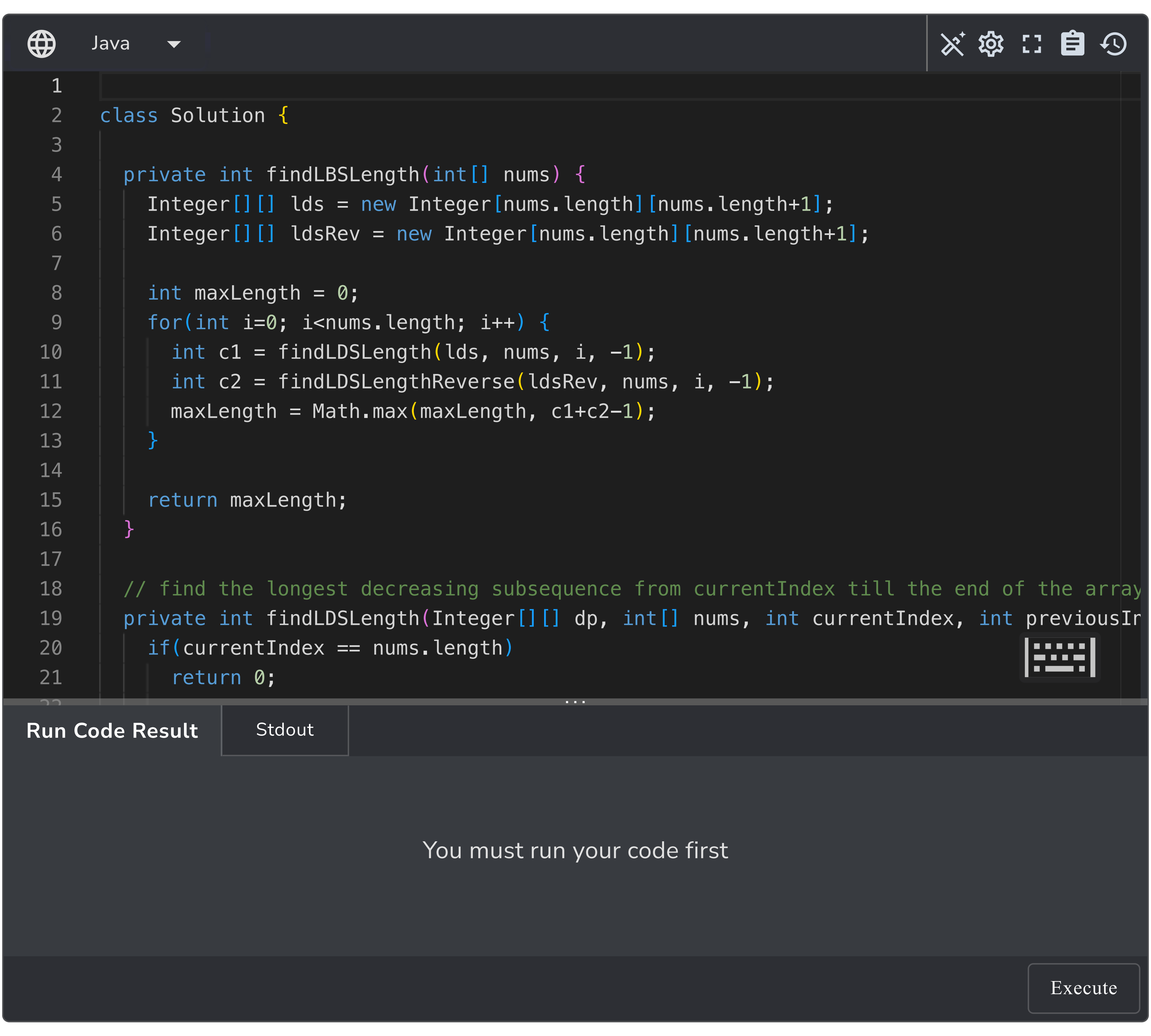
Top-down Dynamic Programming with Memoization

To overcome the overlapping subproblems, we can use an array to store the already solved subproblems.

We need to memoize the recursive functions that calculate the longest decreasing subsequence. The two changing values for our recursive function are the current and the previous index. Therefore, we can store the results of all subproblems in a two-dimensional array. (Another alternative could be to use a hash-table whose key would be a string (currentIndex + "|" + previousIndex)).

Code

Here is the code:



```java

class Solution {

  private int findLBSLength(int[] nums) {
    Integer[][] lds = new Integer[nums.length][nums.length+1];
    Integer[][] ldsRev = new Integer[nums.length][nums.length+1];

    int maxLength = 0;
    for(int i=0; i<nums.length; i++) {
      int c1 = findLDSLength(lds, nums, i, -1);
      int c2 = findLDSLengthReverse(ldsRev, nums, i, -1);
      maxLength = Math.max(maxLength, c1+c2-1);
    }

    return maxLength;
  }

  // find the longest decreasing subsequence from currentIndex till the end of the array
  private int findLDSLength(Integer[][] dp, int[] nums, int currentIndex, int previousIr
    if(currentIndex == nums.length)
      return 0;
```

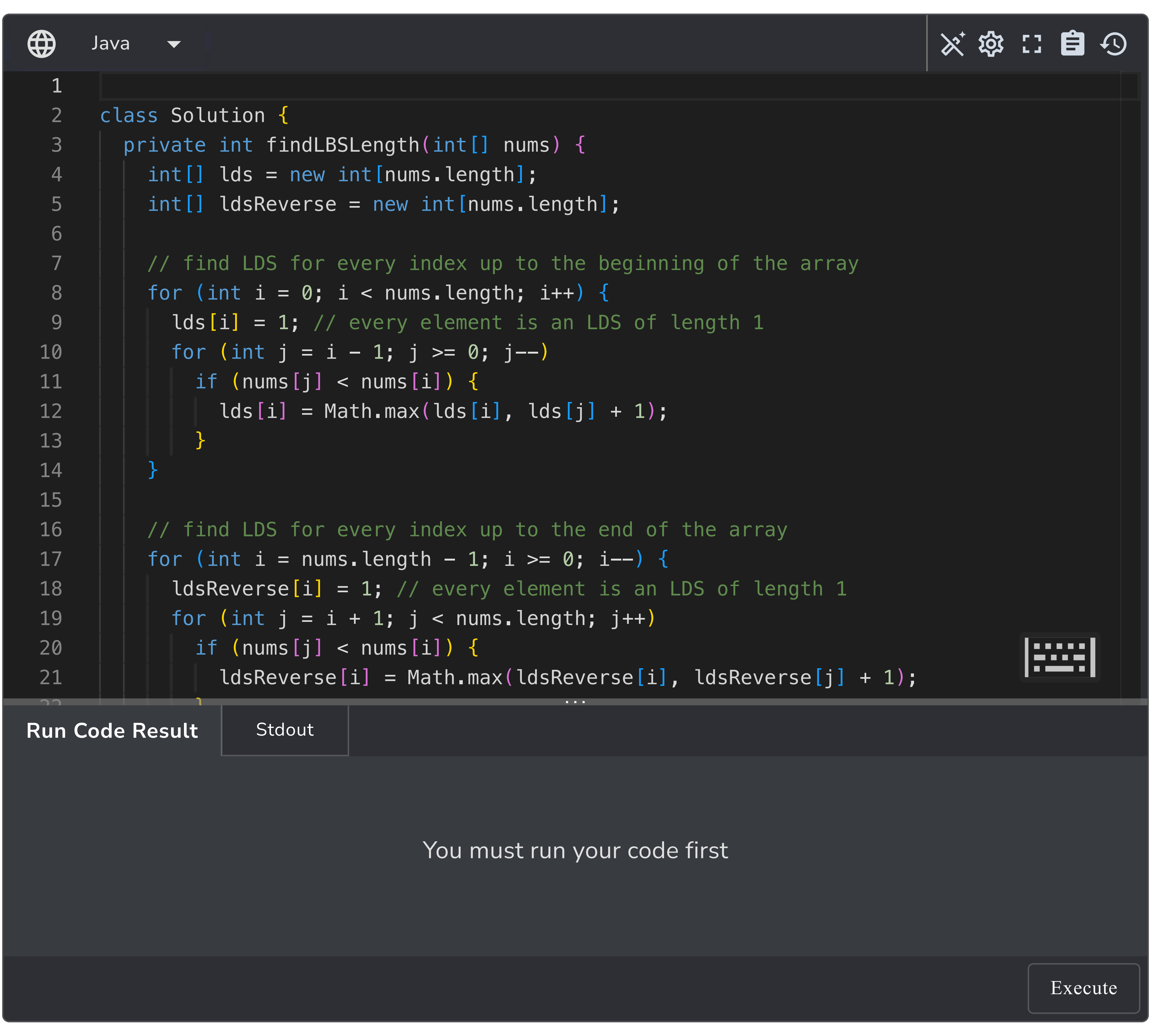
You must run your code first

Bottom-up Dynamic Programming

The above algorithm shows us a clear bottom-up approach. We can separately calculate LDS for every index i.e., from the beginning to the end of the array and vice versa. The required length of LBS would be the one that has the maximum sum of LDS for a given index (from both ends).

Code

Here is the code for our bottom-up dynamic programming approach:



```java

class Solution {
  private int findLBSLength(int[] nums) {
    int[] lds = new int[nums.length];
    int[] ldsReverse = new int[nums.length];

    // find LDS for every index up to the beginning of the array
    for (int i = 0; i < nums.length; i++) {
      lds[i] = 1; // every element is an LDS of length 1
      for (int j = i - 1; j >= 0; j--)
        if (nums[j] < nums[i]) {
          lds[i] = Math.max(lds[i], lds[j] + 1);
        }
    }

    // find LDS for every index up to the end of the array
    for (int i = nums.length - 1; i >= 0; i--) {
      ldsReverse[i] = 1; // every element is an LDS of length 1
      for (int j = i + 1; j < nums.length; j++)
        if (nums[j] < nums[i]) {
          ldsReverse[i] = Math.max(ldsReverse[i], ldsReverse[j] + 1);
```

You must run your code first

The time complexity of the above algorithm is $O(n^2)$ and the space complexity is $O(n)$.

← Previous
Longest Bitonic Subsequence

Next →
Longest Alternating Subsequence

☐ Mark as Completed